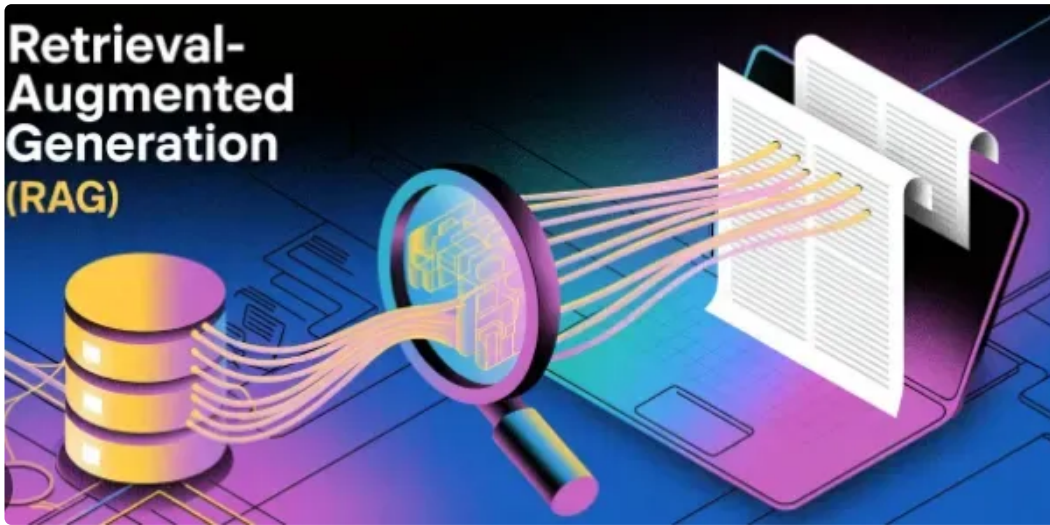


RAG系统完整指南

SuperAI 机器之魂 2026年2月6日 06:09 美国

字数 23445, 阅读大约需 63 分钟



引言

检索增强生成（Retrieval-Augmented Generation，简称RAG）通过将大型语言模型的能力与外部知识检索相结合，彻底改变了我们构建智能系统的方式。当组织面临大语言模型产生幻觉、知识过时以及特定领域知识缺失等问题时，RAG已成为将AI响应锚定在事实性、最新信息中的事实标准解决方案。

什么是RAG及其重要性

根本性问题

大型语言模型在静态数据集上进行训练，存在知识截止日期的限制。它们存在以下局限性：

- 无法访问实时信息
- 在不确定时会产生幻觉
- 缺乏特定领域的知识

- 无法有效引用来源
- 难以处理专有数据/私有数据

RAG解决方案

RAG通过以下方式解决这些限制：

查询 → 检索相关上下文 → 增强提示词 → 生成响应

核心优势：

- **事实准确性**：将响应锚定在检索到的文档中
- **来源归属**：提供引用和参考
- **动态知识**：访问当前信息
- **领域专业知识**：与专业知识库集成
- **减少幻觉**：上下文约束生成
- **成本效益**：较小模型配合外部记忆

2. RAG核心组件

每个RAG系统都由四个基本组件组成：

2.1 文档处理流水线

原始文档 → 分块 → 嵌入 → 向量存储

分块策略：

- 固定大小分块（例如512个token）
- 语义分块（基于段落/章节）
- 滑动窗口重叠
- 递归结构拆分

嵌入模型：

- OpenAI `text-embedding-3-large` (3072维)
- Cohere `embed-english-v3.0` (1024维)
- Sentence Transformers `all-MiniLM-L6-v2` (384维)
- BGE `bge-large-en-v1.5` (1024维)

2.2 向量数据库

存储嵌入向量以进行相似性搜索：

- **Pinecone**：托管式、可扩展、低延迟
- **Weaviate**：开源、混合搜索
- **Chroma**：轻量级、开发者友好
- **Qdrant**：高性能、基于Rust
- **Milvus**：分布式、企业级
- **FAISS**：内存级、Facebook的库

2.3 检索机制

根据查询相似度获取相关文档：

- **稠密检索**：向量相似度（余弦相似度、点积）
- **稀疏检索**：BM25、TF-IDF
- **混合检索**：结合稠密+稀疏
- **重排序**：交叉编码器模型提升精度

2.4 生成组件

使用检索到的上下文生成响应的LLM：

- GPT-4、Claude、Gemini（闭源）
- Llama 3、Mistral、Mixtral（开源）

- 领域特定微调模型

3. RAG架构类型

让我详细解析**10种不同的RAG架构**，从基础到前沿：

3.1 基础RAG（Naive RAG）

基础架构：简单的检索-然后-生成流水线。

用户查询 → 嵌入查询 → 向量搜索 → Top-K文档 → LLM提示词 → 响应

特点：

- 单步检索
- 无查询优化
- 直接Top-K选择
- 最简单的实现

适用场景：

- FAQ系统
- 简单文档搜索
- MVP/原型开发
- 低复杂度知识库

局限性：

- 对模糊查询精度较差
- 无上下文感知
- 难以处理复杂问题
- 对分块质量敏感

3.2 高级RAG

增强流水线： 添加检索前和检索后优化。

查询 → 查询重写 → 多查询 → 检索 → 重排序 → 上下文压缩 → LLM

关键增强：

检索前：

- 查询重写/扩展
- 假设文档嵌入（HyDE）
- 查询分解

检索后：

- 使用交叉编码器重排序
- 上下文压缩
- 相关性过滤

适用场景：

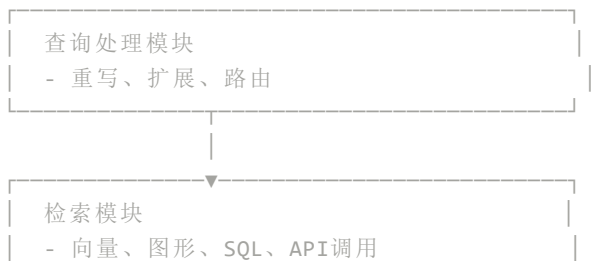
- 复杂问答系统
- 研究助手
- 法律/医疗文档分析
- 处理细微查询的客户支持

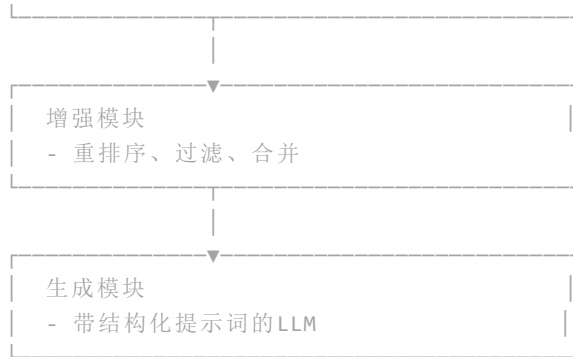
相对于基础RAG的改进：

- 检索精度提升30%-50%
- 减少上下文噪声
- 更好地处理模糊查询

3.3 模块化RAG

基于组件的架构： 混合搭配检索、生成和增强模块。





关键特性：

- 可插拔组件
- 多检索源
- 自定义增强逻辑
- 对不同模块进行A/B测试

适用场景：

- 企业RAG系统
- 多源知识集成
- 实验性RAG研究
- 自定义领域应用

3.4 智能体RAG（Agentic RAG）

自主检索： LLM智能体决定何时以及检索什么。

查询 → 智能体规划 → [决策循环：检索？生成？搜索？] → 响应

智能体能力：

- 判断是否需要检索
- 制定搜索查询
- 迭代检索
- 自我批评和优化
- 使用工具（搜索、计算器、SQL）

框架：

- 带React智能体的LangGraph

- CrewAI多智能体系统
- AutoGPT风格的自主智能体

适用场景：

- 复杂研究任务
- 多步骤推理问题
- 动态信息需求
- 探索性数据分析

示例流程：

用户： "比较所有部门的第四季度收入并识别趋势"

智能体：

1. 检索第四季度财务数据
2. 意识到需要部门映射
3. 检索组织结构
4. 计算趋势
5. 生成比较分析

3.5 分层RAG

多级检索： 文档按层次结构组织。



检索策略：

1. 粗粒度检索（文档级别）
2. 细粒度检索（分块级别）
3. 从父节点扩展上下文

适用场景：

- 长篇文档（书籍、手册）
- 技术文档
- 带章节的法律合同
- 带层次结构的学术论文

优势：

- 保持文档结构
- 更好地保留上下文
- 对大文档高效
- 减少碎片化问题

3.6 图RAG

知识图谱集成：从图结构化知识中检索。

查询 → 实体提取 → 图遍历 → 子图检索 → LLM

组件：

- **节点：**实体（人、地点、概念）
- **边：**实体之间的关系
- **属性：**节点/边的属性

图数据库：

- Neo4j
- Amazon Neptune
- TigerGraph
- OrientDB

适用场景：

- 关系密集型领域（社交网络、组织架构图）
- 多跳推理
- 因果推断
- 知识发现

示例：

查询："Alice如何与OpenAI项目关联？"

图遍历：

Alice → 工作于 → 公司X → 合作伙伴 → OpenAI

Alice → 导师 → Bob → 创立 → OpenAI（间接）

3.7 混合RAG

多模态检索：结合稠密向量 + 稀疏关键词 + 元数据。



融合方法：

- **倒数排名融合（RRF）：** $score = \sum (1/(k + rank_i))$
- **线性组合：** $\alpha * \text{稠密分数} + \beta * \text{稀疏分数}$
- **学习融合：** ML模型组合分数

适用场景：

- 电子商务搜索（语义+精确匹配）
- 医学文献（术语+概念）
- 法律发现（引用+语义含义）

性能：

- 比纯稠密方法提升15%-25%
- 同时处理语义和关键词查询
- 对词汇不匹配更鲁棒

3.8 纠正RAG（CRAG）

自我纠正流水线：评估检索质量并纠正错误。

查询 → 检索 → 质量检查 → [良好? → 生成 | 差? → 网络搜索/重写]

质量评估：

- 使用轻量级模型进行相关性评分
- 置信度阈值

- 矛盾检测

纠正策略：

1. **网络搜索回退**：如果本地检索失败
2. **查询优化**：重写并重试
3. **文档过滤**：移除低置信度分块
4. **知识提取**：仅提取相关部分

适用场景：

- 高风险应用（医疗、法律）
- 动态知识领域
- 事实核查系统
- 需要验证信息的情况

3.9 自反思RAG（Self-RAG）

自我反思生成：模型批评自己的输出并根据需要检索。

生成Token → 自我反思 → [需要证据？ → 检索 → 生成]

反思类型：

- **检索必要性**：我需要检索更多上下文吗？
- **相关性**：这篇检索到的文档相关吗？
- **支持**：证据支持我的说法吗？
- **有用性**：这个输出有帮助吗？

特殊Token：

[Retrieve] - 触发检索
[IsRel] - 相关性检查
[IsSup] - 支持验证
[IsUse] - 效用评估

适用场景：

- 长篇内容生成
- 研究综合
- 事实密集型写作

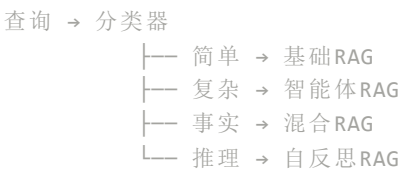
- 需要高事实准确性的情况

训练：

- 需要专门微调的模型
- 使用来自反思的强化学习

3.10 自适应RAG

查询自适应路由：根据查询类型动态选择RAG策略。



分类维度：

- 查询复杂度（简单、中等、复杂）
- 信息需求（事实、分析、创意）
- 领域特定性
- 延迟要求

路由策略：

- 基于规则：关键词/模式匹配
- 基于ML：查询分类器模型
- 基于LLM：少样本分类

适用场景：

- 多用途聊天机器人
- 企业知识系统
- 成本优化（尽可能使用更简单的RAG）
- SLA驱动的应用

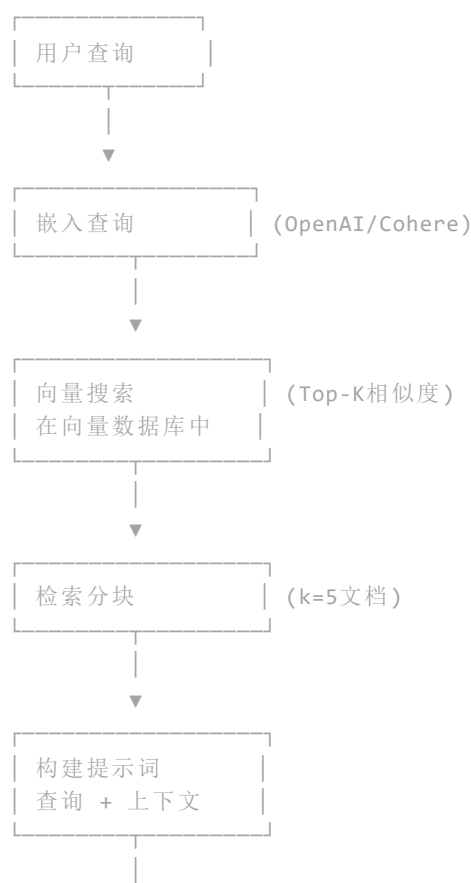
4. 详细对比矩阵

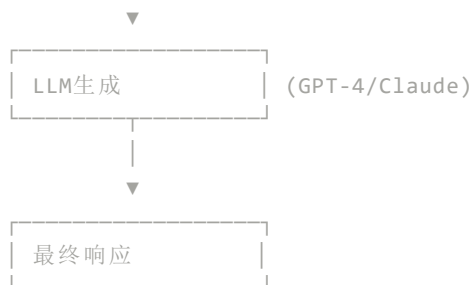
指标说明：

- **延迟**：平均端到端响应时间
- **精度**：检索精度@k=5
- **成本**：相对运营成本（计算+API）

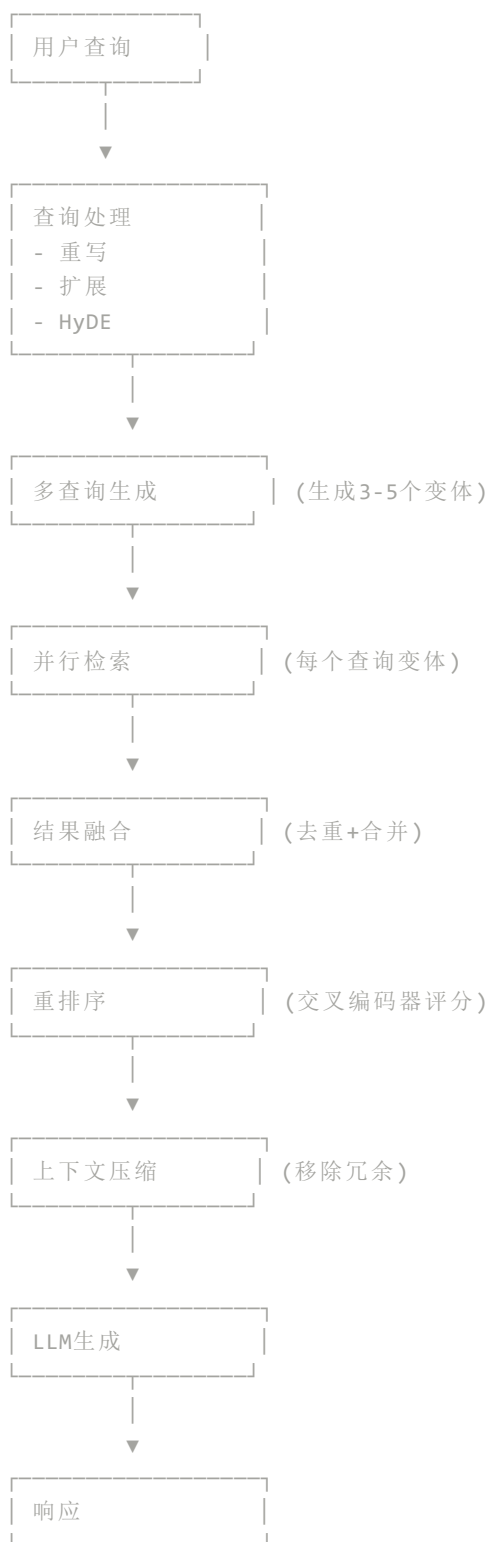
5. 每种架构的流程图

5.1 基础RAG流程

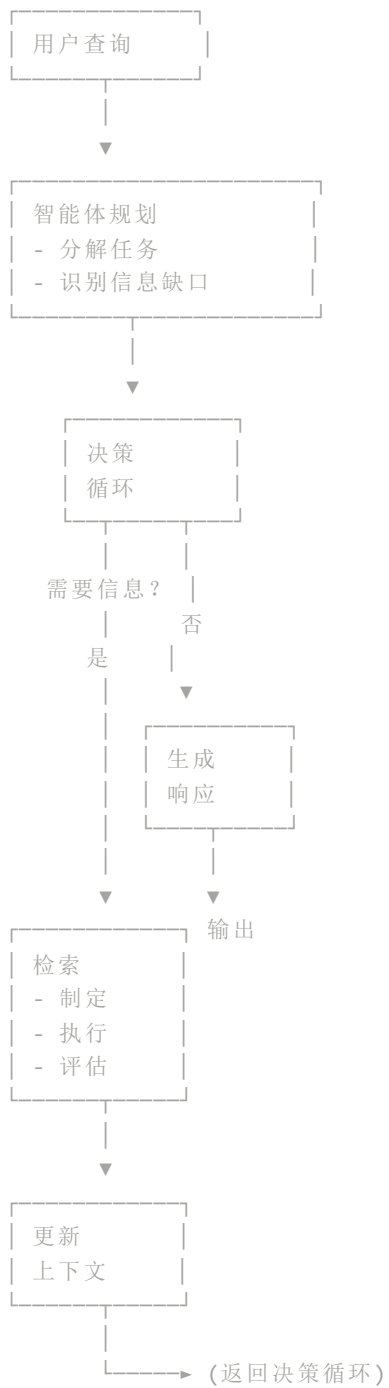




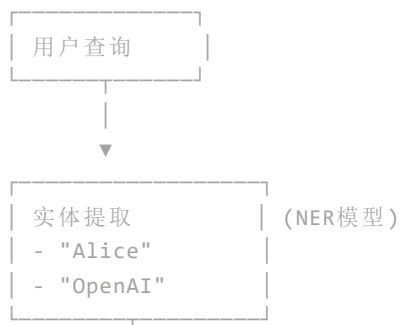
5.2 高级RAG流程

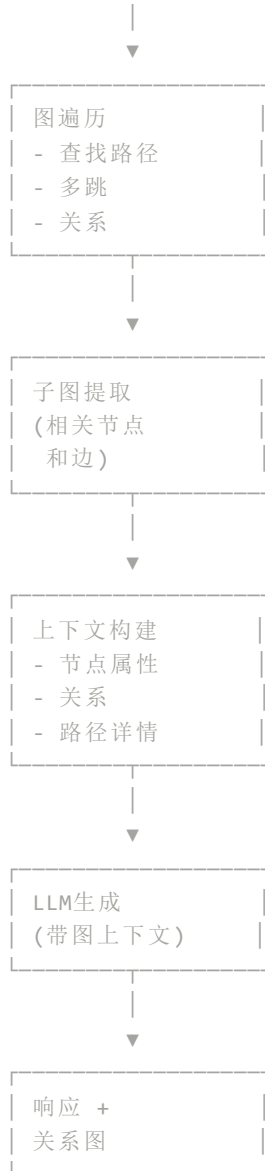


5.3 智能体RAG流程

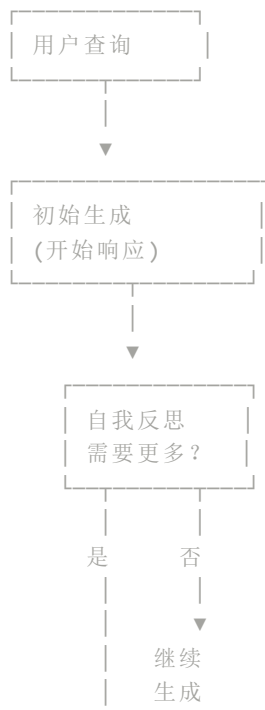


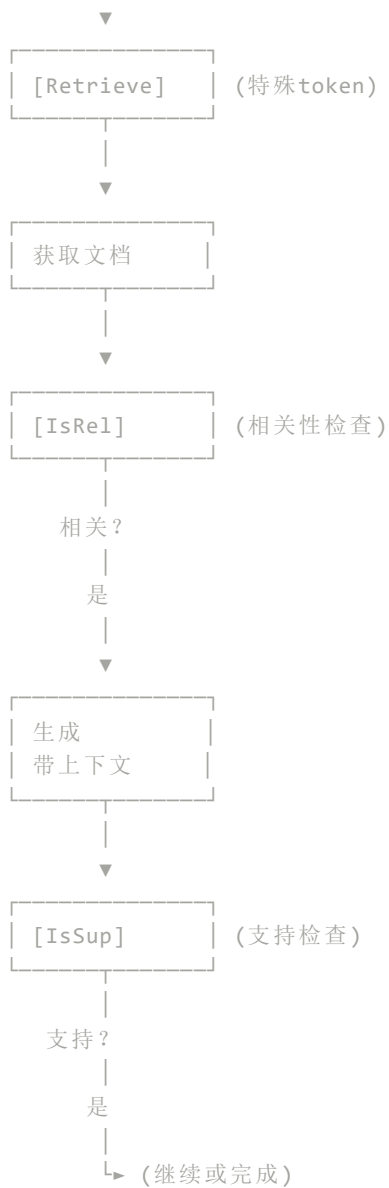
5.4 图RAG流程





5.5 自反思RAG流程





6. 生产级代码实现

6.1 基础RAG实现

```
# naive_rag.py
# 使用最新库的生产级基础RAG
# 依赖: openai==1.58.1, chromadb==0.5.23, langchain==0.3.13

import os
from typing import List, Dict
import chromadb
from chromadb.config import Settings
from openai import OpenAI
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import TextLoader, PyPDFLoader
import json
```



```

class NaiveRAG:
    """
    生产级基础RAG实现。
    简单的检索-然后-生成流水线。
    """

    def __init__(
        self,
        collection_name: str = "naive_rag_docs",
        embedding_model: str = "text-embedding-3-large",
        llm_model: str = "gpt-4-turbo-preview",
        chunk_size: int = 1000,
        chunk_overlap: int = 200,
        top_k: int = 5
    ):
        self.openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
        self.embedding_model = embedding_model
        self.llm_model = llm_model
        self.top_k = top_k

        # 初始化ChromaDB
        self.chroma_client = chromadb.Client(Settings(
            anonymized_telemetry=False,
            allow_reset=True
        ))

        # 创建或获取集合
        self.collection = self.chroma_client.get_or_create_collection(
            name=collection_name,
            metadata={"hnsw:space": "cosine"}
        )

        # 初始化文本分割器
        self.text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=chunk_size,
            chunk_overlap=chunk_overlap,
            separators=["\n\n", "\n", ". ", " ", ""]
        )

    def embed_text(self, text: str) -> List[float]:
        """使用OpenAI API生成嵌入。"""
        response = self.openai_client.embeddings.create(
            model=self.embedding_model,
            input=text
        )
        return response.data[0].embedding

    def ingest_documents(self, file_paths: List[str]) -> Dict[str, int]:
        """
        将文档摄取到向量数据库。
        支持.txt和.pdf文件。
        """
        total_chunks = 0

        for file_path in file_paths:
            # 加载文档
            if file_path.endswith('.pdf'):
                loader = PyPDFLoader(file_path)
            else:
                loader = TextLoader(file_path)

            documents = loader.load()

            # 分割成块
            chunks = self.text_splitter.split_documents(documents)

```

```

# 准备ChromaDB数据
texts = [chunk.page_content for chunk in chunks]
embeddings = [self.embed_text(text) for text in texts]
ids = [f"{file_path}_{i}" for i in range(len(chunks))]
metadatas = [
    {
        "source": file_path,
        "chunk_index": i,
        "char_count": len(chunk.page_content)
    }
    for i, chunk in enumerate(chunks)
]

# 添加到集合
self.collection.add(
    embeddings=embeddings,
    documents=texts,
    ids=ids,
    metadatas=metadatas
)

total_chunks += len(chunks)
print(f"✓ 已摄取 {file_path}: {len(chunks)} 个块")

return {"total_documents": len(file_paths), "total_chunks": total_chunks}

def retrieve(self, query: str) -> List[Dict]:
    """检索top-k相关文档。"""
    query_embedding = self.embed_text(query)

    results = self.collection.query(
        query_embeddings=[query_embedding],
        n_results=self.top_k,
        include=["documents", "metadatas", "distances"]
    )

    retrieved_docs = []
    for i in range(len(results['documents'][0])):
        retrieved_docs.append({
            "content": results['documents'][0][i],
            "metadata": results['metadatas'][0][i],
            "score": 1 - results['distances'][0][i] # 将距离转换为相似度
        })

    return retrieved_docs

def generate(self, query: str, context_docs: List[Dict]) -> Dict:
    """使用检索到的上下文生成响应。"""
    # 构建上下文字符串
    context = "\n\n".join([
        f"[来源 {i+1}]: {doc['content']}"
        for i, doc in enumerate(context_docs)
    ])

    # 构建提示词
    prompt = f"""你是一个有帮助的助手，根据提供的上下文回答问题。

```

上下文:

{context}

问题: {query}

指令:

- 仅使用以上上下文中的信息回答问题

- 如果上下文没有包含足够信息，请说明
- 使用[来源 N]引用
- 简洁准确

答案："""

```

        # 生成响应
        response = self.openai_client.chat.completions.create(
            model=self.llm_model,
            messages=[\
                {"role": "system", "content": "你是一个有帮助的助手。"},\
                {"role": "user", "content": prompt}\
            ],
            temperature=0.3,
            max_tokens=1000
        )

        return {
            "answer": response.choices[0].message.content,
            "sources": [doc['metadata'] for doc in context_docs],
            "usage": response.usage.model_dump()
        }

def query(self, question: str) -> Dict:
    """
    主RAG流水线：检索 + 生成。
    """
    # 检索相关文档
    retrieved_docs = self.retrieve(question)

    # 生成答案
    result = self.generate(question, retrieved_docs)

    return {
        "question": question,
        "answer": result["answer"],
        "retrieved_documents": retrieved_docs,
        "sources": result["sources"],
        "usage": result["usage"]
    }

# 示例用法
if __name__ == "__main__":
    # 初始化RAG系统
    rag = NaiveRAG(
        collection_name="my_knowledge_base",
        top_k=5
    )

    # 摄取文档
    stats = rag.ingest_documents([\
        "company_handbook.pdf",\
        "technical_docs.txt"\
    ])
    print(f"\n摄取完成: {stats}")

    # 查询系统
    response = rag.query(
        "我们的远程工作政策是什么？"
    )

    print(f"\n问题: {response['question']}")
    print(f"\n答案: {response['answer']}")
    print(f"\n使用的来源: {len(response['sources'])}")
    print(f"使用的token: {response['usage']}")

```

6.2 高级RAG实现

```
# advanced_rag.py
# 带查询重写、多查询和重排序的生产级高级RAG
# 依赖: langchain==0.3.13, sentence-transformers==3.3.1, cohere==5.11.4

import os
from typing import List, Dict
import chromadb
from openai import OpenAI
import cohere
from sentence_transformers import CrossEncoder
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import TextLoader, PyPDFLoader
import numpy as np

class AdvancedRAG:
    """
    高级RAG，包含：
    - 查询重写
    - 多查询生成
    - HyDE（假设文档嵌入）
    - 交叉编码器重排序
    - 上下文压缩
    """

    def __init__(
        self,
        collection_name: str = "advanced_rag_docs",
        embedding_model: str = "text-embedding-3-large",
        llm_model: str = "gpt-4-turbo-preview",
        reranker_model: str = "cross-encoder/ms-marco-MiniLM-L-12-v2",
        chunk_size: int = 800,
        chunk_overlap: int = 150,
        initial_k: int = 20,
        final_k: int = 5
    ):
        self.openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
        self.cohere_client = cohere.Client(os.getenv("COHERE_API_KEY"))
        self.embedding_model = embedding_model
        self.llm_model = llm_model
        self.initial_k = initial_k
        self.final_k = final_k

        # 初始化ChromaDB
        self.chroma_client = chromadb.Client()
        self.collection = self.chroma_client.get_or_create_collection(
            name=collection_name,
            metadata={"hnsw:space": "cosine"}
        )

        # 初始化重排序器
        self.reranker = CrossEncoder(reranker_model, max_length=512)

        # 初始化文本分割器
        self.text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=chunk_size,
            chunk_overlap=chunk_overlap,
            separators=["\n\n", "\n", ". ", " ", ""]
        )

    def embed_text(self, text: str) -> List[float]:
```

```

"""使用OpenAI生成嵌入。"""
response = self.openai_client.embeddings.create(
    model=self.embedding_model,
    input=text
)
return response.data[0].embedding

def ingest_documents(self, file_paths: List[str]) -> Dict:
    """摄取带元数据的文档。"""
    total_chunks = 0

    for file_path in file_paths:
        if file_path.endswith('.pdf'):
            loader = PyPDFLoader(file_path)
        else:
            loader = TextLoader(file_path)

        documents = loader.load()
        chunks = self.text_splitter.split_documents(documents)

        texts = [chunk.page_content for chunk in chunks]
        embeddings = [self.embed_text(text) for text in texts]
        ids = [f"{file_path}_{i}" for i in range(len(chunks))]
        metadatas = [
            {
                "source": file_path,
                "chunk_index": i,
                "char_count": len(chunk.page_content)
            }
            for i, chunk in enumerate(chunks)
        ]

        self.collection.add(
            embeddings=embeddings,
            documents=texts,
            ids=ids,
            metadatas=metadatas
        )

        total_chunks += len(chunks)

    return {"total_documents": len(file_paths), "total_chunks": total_chunks}

def rewrite_query(self, query: str) -> str:
    """重写查询以获得更好的检索效果。"""
    prompt = f"""重写以下查询，使其更具体、更适合检索。
专注于关键概念并展开缩写。

原始查询: {query}

重写后的查询: """

```

```

response = self.openai_client.chat.completions.create(
    model=self.llm_model,
    messages=[
        {"role": "system", "content": "你是查询优化专家。"},
        {"role": "user", "content": prompt}
    ],
    temperature=0.3,
    max_tokens=100
)

return response.choices[0].message.content.strip()

```

```

def generate_multi_queries(self, query: str, n: int = 3) -> List[str]:

```

```
"""生成多个查询变体。"""
```

```
prompt = f"""生成以下查询的{n}个不同变体。
```

每个变体应该从不同角度处理问题。

原始查询: {query}

仅返回变体，每行一个，编号1-{n}: """

```
response = self.openai_client.chat.completions.create(
    model=self.llm_model,
    messages=[\
        {"role": "system", "content": "你是查询扩展专家。"},\
        {"role": "user", "content": prompt}\
    ],
    temperature=0.7,
    max_tokens=200
)

variations = response.choices[0].message.content.strip().split('\n')
# 清理编号列表
queries = [q.split('. ', 1)[-1].strip() for q in variations if q.strip()]
return queries[:n]

def hyde_generate(self, query: str) -> str:
    """
    假设文档嵌入（HyDE）。
    生成假设答案，然后使用它进行检索。
    """
    prompt = f"""假设你有相关文档，生成一个简洁、事实性的答案来回答这个问题。
```

问题: {query}

假设答案: """

```
response = self.openai_client.chat.completions.create(
    model=self.llm_model,
    messages=[\
        {"role": "system", "content": "你是一个知识渊博的助手。"},\
        {"role": "user", "content": prompt}\
    ],
    temperature=0.5,
    max_tokens=200
)

return response.choices[0].message.content.strip()

def retrieve_with_fusion(self, queries: List[str]) -> List[Dict]:
    """
    为多个查询检索文档并融合结果。
    使用倒数排名融合（RRF）。
    """
    all_results = {}

    for query in queries:
        query_embedding = self.embed_text(query)
        results = self.collection.query(
            query_embeddings=[query_embedding],
            n_results=self.initial_k,
            include=["documents", "metadatas", "distances"]
        )

        # 添加带RRF评分的结果
        for i, doc_id in enumerate(results['ids'][0]):
            rank = i + 1
            rrf_score = 1 / (60 + rank) # k=60是标准值
```

```

        if doc_id not in all_results:
            all_results[doc_id] = {
                "content": results['documents'][0][i],
                "metadata": results['metadatas'][0][i],
                "rrf_score": 0,
                "appearances": 0
            }

        all_results[doc_id]["rrf_score"] += rrf_score
        all_results[doc_id]["appearances"] += 1

# 按RRF分数排序
sorted_results = sorted(
    all_results.values(),
    key=lambda x: x["rrf_score"],
    reverse=True
)

return sorted_results[:self.initial_k]

def rerank(self, query: str, documents: List[Dict]) -> List[Dict]:
    """使用交叉编码器重排序文档。"""
    if not documents:
        return documents

    # 准备重排序对
    pairs = [[query, doc["content"]] for doc in documents]

    # 获取重排序分数
    scores = self.reranker.predict(pairs)

    # 将分数添加到文档
    for doc, score in zip(documents, scores):
        doc["rerank_score"] = float(score)

    # 按重排序分数排序
    reranked = sorted(documents, key=lambda x: x["rerank_score"], reverse=True)

    return reranked[:self.final_k]

def compress_context(self, query: str, documents: List[Dict]) -> List[Dict]:
    """
    使用Cohere的重排序API压缩上下文，该API还提供相关性评分用于过滤。
    """
    if not documents:
        return documents

    # 使用Cohere进行压缩和相关性评分
    results = self.cohere_client.rerank(
        model="rerank-english-v3.0",
        query=query,
        documents=[doc["content"] for doc in documents],
        top_n=self.final_k,
        return_documents=True
    )

    compressed_docs = []
    for result in results.results:
        compressed_docs.append({
            "content": result.document.text,
            "metadata": documents[result.index]["metadata"],
            "relevance_score": result.relevance_score
        })

```

```
return compressed_docs
```

```
def generate(self, query: str, context_docs: List[Dict]) -> Dict:
    """使用压缩上下文生成最终答案。"""
    context = "\n\n".join([\
        f"[来源 {i+1}] (相关性: {doc.get('relevance_score', doc.get('rerank_score'
        for i, doc in enumerate(context_docs)\
    ])

    prompt = f"""你是一个有帮助的助手。使用提供的上下文回答问题。
```

上下文:

{context}

问题: {query}

指令:

- 仅使用上下文中的信息
- 用[来源 N]引用来源
- 如果信息不足，清楚地说明
- 精确简洁

答案: """

```
response = self.openai_client.chat.completions.create(
    model=self.llm_model,
    messages=[\
        {"role": "system", "content": "你是一个有帮助、准确的助手。"},\
        {"role": "user", "content": prompt}\
    ],
    temperature=0.3,
    max_tokens=1000
)

return {
    "answer": response.choices[0].message.content,
    "sources": [doc['metadata'] for doc in context_docs],
    "usage": response.usage.model_dump()
}
```

```
def query(self, question: str, use_hyde: bool = False) -> Dict:
    """
```

主高级RAG流水线。

步骤:

1. 查询重写
2. 多查询生成
3. 可选HyDE
4. 融合检索
5. 重排序
6. 上下文压缩
7. 生成

"""

```
print("🔍 使用高级RAG处理查询...")
```

```
# 步骤1: 重写查询
```

```
print("    └─ 重写查询...")
```

```
rewritten_query = self.rewrite_query(question)
```

```
# 步骤2: 生成查询变体
```

```
print("    └─ 生成查询变体...")
```

```
query_variations = self.generate_multi_queries(rewritten_query, n=3)
```

```
# 步骤3: 可选HyDE
```

```
if use_hyde:
```



```

        print("    └─ 生成假设文档 (HyDE) ...")
        hyde_doc = self.hyde_generate(question)
        query_variations.append(hyde_doc)

    all_queries = [rewritten_query] + query_variations

    # 步骤4: 融合检索
    print(f"    └─ 为{len(all_queries)}个查询检索文档...")
    fused_results = self.retrieve_with_fusion(all_queries)

    # 步骤5: 重排序
    print(f"    └─ 重排序{len(fused_results)}个文档...")
    reranked_docs = self.rerank(question, fused_results)

    # 步骤6: 压缩上下文
    print("    └─ 压缩上下文...")
    compressed_docs = self.compress_context(question, reranked_docs)

    # 步骤7: 生成
    print("    └─ 生成最终答案...")
    result = self.generate(question, compressed_docs)

    return {
        "question": question,
        "rewritten_query": rewritten_query,
        "query_variations": query_variations,
        "answer": result["answer"],
        "retrieved_documents": compressed_docs,
        "sources": result["sources"],
        "usage": result["usage"]
    }

}

# 示例用法
if __name__ == "__main__":
    rag = AdvancedRAG(
        collection_name="advanced_kb",
        initial_k=20,
        final_k=5
    )

    # 摄取文档
    stats = rag.ingest_documents([
        "technical_manual.pdf",
        "faq_document.txt"
    ])
    print(f"摄取: {stats}")

    # 使用高级流水线查询
    response = rag.query(
        "如何配置数据库连接?",
        use_hyde=True
    )

    print(f"\n{'='*60}")
    print(f"问题: {response['question']}")
    print(f"重写后: {response['rewritten_query']}")
    print(f"\n答案:\n{response['answer']}")
    print(f"\n来源: {len(response['sources'])}")

```

6.3 智能体RAG实现

```
# agentic_rag.py
# 使用LangGraph的生产级智能体RAG
# 依赖: langgraph==0.2.55, langchain==0.3.13, tavily-python==0.5.0

import os
from typing import TypedDict, Annotated, List, Dict
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolExecutor
from langchain_openai import ChatOpenAI
from langchain.tools import Tool
from langchain.prompts import ChatPromptTemplate
from langchain_community.tools.tavily_search import TavilySearchResults
import chromadb
from openai import OpenAI

class AgentState(TypedDict):
    """RAG智能体的状态。"""
    question: str
    plan: str
    retrieved_docs: List[Dict]
    web_results: List[Dict]
    thoughts: List[str]
    answer: str
    iterations: int
    max_iterations: int

class AgenticRAG:
    """
    智能体RAG，具有自主决策能力。

    智能体能力：
    - 决定何时从本地知识库检索
    - 决定何时搜索网络
    - 规划多步骤检索策略
    - 自我批评和优化
    """

    def __init__(
        self,
        collection_name: str = "agentic_rag_docs",
        llm_model: str = "gpt-4-turbo-preview",
        max_iterations: int = 5
    ):
        self.openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
        self.llm = ChatOpenAI(model=llm_model, temperature=0)
        self.max_iterations = max_iterations

        # 初始化ChromaDB
        self.chroma_client = chromadb.Client()
        self.collection = self.chroma_client.get_or_create_collection(
            name=collection_name
        )

        # 初始化工具
        self.tools = self._initialize_tools()
        self.tool_executor = ToolExecutor(self.tools)

        # 构建智能体图
        self.graph = self._build_graph()
```

```

def _initialize_tools(self) -> List[Tool]:
    """初始化智能体工具。"""

    # 本地检索工具
    def local_retrieve(query: str) -> str:
        """从本地知识库检索。"""
        embedding = self.embed_text(query)
        results = self.collection.query(
            query_embeddings=[embedding],
            n_results=5,
            include=["documents", "metadatas"]
        )

        if not results['documents'][0]:
            return "在本地知识库中未找到相关文档。"

        docs_text = "\n\n".join([
            f"文档 {i+1}:\n{doc}"
            for i, doc in enumerate(results['documents'][0])
        ])
        return docs_text

    # 网络搜索工具
    web_search = TavilySearchResults(
        max_results=5,
        api_key=os.getenv("TAVILY_API_KEY")
    )

    return [
        Tool(
            name="LocalRetrieval",
            func=local_retrieve,
            description="从本地知识库检索信息。用于公司特定或已摄取的领域特定信息。"
        ),
        Tool(
            name="WebSearch",
            func=web_search.invoke,
            description="搜索网络获取当前信息、事实或本地知识库中没有的主题。用于一般性信息。"
        )
    ]

def embed_text(self, text: str) -> List[float]:
    """生成嵌入。"""
    response = self.openai_client.embeddings.create(
        model="text-embedding-3-large",
        input=text
    )
    return response.data[0].embedding

def _build_graph(self) -> StateGraph:
    """构建智能体工作流图。"""
    workflow = StateGraph(AgentState)

    # 添加节点
    workflow.add_node("planner", self.plan_node)
    workflow.add_node("retriever", self.retrieve_node)
    workflow.add_node("evaluator", self.evaluate_node)
    workflow.add_node("generator", self.generate_node)

    # 添加边
    workflow.set_entry_point("planner")
    workflow.add_edge("planner", "retriever")
    workflow.add_edge("retriever", "evaluator")

    # 条件边：继续检索或生成

```

```

        workflow.add_conditional_edges(
            "evaluator",
            self.should_continue,
            {
                "continue": "retriever",
                "generate": "generator"
            }
        )
        workflow.add_edge("generator", END)

    return workflow.compile()

def plan_node(self, state: AgentState) -> AgentState:
    """智能体规划检索策略。"""
    plan_prompt = ChatPromptTemplate.from_template("""
你是智能RAG智能体。分析问题并创建检索计划。

问题: {question}

确定:
1. 这是否需要本地知识库检索? (是/否)
2. 这是否需要网络搜索? (是/否)
3. 我们在寻找什么具体信息?
4. 搜索策略是什么?

提供简洁计划:
""")

    messages = plan_prompt.format_messages(question=state["question"])
    response = self.llm.invoke(messages)

    state["plan"] = response.content
    state["thoughts"] = [f"计划: {response.content}"]
    state["iterations"] = 0
    state["max_iterations"] = self.max_iterations

    return state

def retrieve_node(self, state: AgentState) -> AgentState:
    """根据计划执行检索。"""
    retrieve_prompt = ChatPromptTemplate.from_template("""
问题: {question}
当前计划: {plan}
迭代: {iterations}/{max_iterations}

之前的想法:
{thoughts}

决定使用哪个工具以及执行什么查询。
按此确切格式响应:
TOOL: [LocalRetrieval|WebSearch]
QUERY: [你的搜索查询]
REASON: [为什么选择这个工具和查询]
""")

    thoughts_text = "\n".join(state["thoughts"])
    messages = retrieve_prompt.format_messages(
        question=state["question"],
        plan=state["plan"],
        iterations=state["iterations"],
        max_iterations=state["max_iterations"],
        thoughts=thoughts_text
    )

    response = self.llm.invoke(messages)

```

```

decision = response.content

# 解析决策
tool_name = None
query = None
for line in decision.split('\n'):
    if line.startswith('TOOL:'):
        tool_name = line.split(':', 1)[1].strip()
    elif line.startswith('QUERY:'):
        query = line.split(':', 1)[1].strip()

if tool_name and query:
    # 执行工具
    tool = next((t for t in self.tools if t.name == tool_name), None)
    if tool:
        result = tool.invoke(query)

        if tool_name == "LocalRetrieval":
            state["retrieved_docs"].append({
                "query": query,
                "result": result
            })
        else:
            state["web_results"].append({
                "query": query,
                "result": result
            })

        state["thoughts"].append(
            f"工具: {tool_name} | 查询: {query} | 结果: {result[:100]}..."
        )

return state

def evaluate_node(self, state: AgentState) -> AgentState:
    """评估检索结果是否足够。"""
    all_results = state["retrieved_docs"] + state["web_results"]
    all_text = "\n".join([r["result"] for r in all_results])

    eval_prompt = f"""
    评估当前信息是否足以回答问题。

    问题: {state["question"]}

    收集的信息:
    {all_text[:2000]}...

    信息是否足够回答问题? 是/否。
    简要说明:
    """

    response = self.llm.invoke(eval_prompt)
    state["thoughts"].append(f"评估: {response.content}")

    return state

def should_continue(self, state: AgentState) -> str:
    """决定是否继续检索。"""
    all_results = state["retrieved_docs"] + state["web_results"]
    all_text = "\n".join([r["result"] for r in all_results])

    # 检查是否达到最大迭代次数
    if state["iterations"] >= state["max_iterations"]:
        return "generate"

```

```

        # 评估信息是否足够
        eval_prompt = f"""
问题: {state["question"]}

当前信息: {all_text[:500]}...

信息是否足够回答问题? 仅回复"是"或"否"。
"""

        response = self.llm.invoke(eval_prompt)
        if "是" in response.content and len(all_results) >= 2:
            return "generate"

        # 否则继续
        return "continue"

def generate_node(self, state: AgentState) -> AgentState:
    """生成最终响应。"""
    all_results = state["retrieved_docs"] + state["web_results"]
    all_text = "\n\n".join([f"[来源 {i+1}]: {r['result']}" for i, r in enumerate(

    prompt = f"""根据检索到的信息回答问题。

```

检索到的信息:

```
{all_text}
```

```
问题: {state["question"]}
```

指令:

- 使用检索到的信息
- 明确标注信息来源
- 如果信息不足, 请说明
- 准确全面

答案: """

```

        response = self.llm.invoke(prompt)
        state["answer"] = response.content
        state["iterations"] += 1

    return state

def query(self, question: str) -> Dict:
    """
    主智能体RAG流水线。
    """
    # 初始化状态
    initial_state = AgentState(
        question=question,
        plan="",
        retrieved_docs=[],
        web_results=[],
        thoughts=[],
        answer="",
        iterations=0,
        max_iterations=self.max_iterations
    )

    # 运行图
    result = self.graph.invoke(initial_state)

    return {
        "question": question,
        "plan": result["plan"],
        "retrieved_docs": result["retrieved_docs"],

```

```

        "web_results": result["web_results"],
        "answer": result["answer"],
        "iterations": result["iterations"]
    }

# 示例用法
if __name__ == "__main__":
    agent_rag = AgenticRAG(
        collection_name="agentic_kb",
        max_iterations=5
    )

    # 查询智能体
    response = agent_rag.query(
        "比较iPhone 15和Samsung Galaxy S24的相机规格"
    )

    print(f"\n{'='*60}")
    print(f"问题: {response['question']}")
    print(f"计划: {response['plan']}")
    print(f"迭代次数: {response['iterations']}")
    print(f"检索的文档数: {len(response['retrieved_docs'])}")
    print(f"网络搜索结果数: {len(response['web_results'])}")
    print(f"最终答案:\n{response['answer']}")

```

6.4 图RAG实现

```

# graph_rag.py
# 带知识图谱的生产级图RAG
# 依赖: langchain==0.3.13, neo4j==5.24.0, openai==1.58.1

import os
from typing import List, Dict, TypedDict
from neo4j import GraphDatabase
from openai import OpenAI
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import TextLoader

class GraphState(TypedDict):
    """图RAG的状态。"""
    question: str
    entities: List[str]
    subgraph: Dict
    documents: List[Dict]
    answer: str

class GraphRAG:
    """
    图RAG，结合知识图谱进行检索。

    特点:
    - 实体提取和关系映射
    - 图遍历用于多跳推理
    - 知识图谱上下文增强生成
    """

    def __init__(

```

```

        self,
        neo4j_uri: str,
        neo4j_user: str,
        neo4j_password: str,
        llm_model: str = "gpt-4-turbo-preview",
        max_hops: int = 2
    ):
        self.driver = GraphDatabase.driver(
            neo4j_uri,
            auth=(neo4j_user, neo4j_password)
        )
        self.openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
        self.llm_model = llm_model
        self.max_hops = max_hops

        # 文本分割器
        self.text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=500,
            chunk_overlap=50
        )

    def close(self):
        """关闭Neo4j连接。"""
        self.driver.close()

```

```

    def extract_entities(self, text: str) -> List[Dict]:
        """
        从文本中提取实体和关系。
        使用LLM进行命名实体识别和关系提取。
        """
        prompt = f"""从以下文本中提取实体及其关系。

```

文本：

```
{text[:2000]}
```

以JSON格式返回：

```

{{
    "entities": [
        {{ "name": "实体名", "type": "类型 (PERSON/ORG/LOCATION/CONCEPT)" }}
    ],
    "relationships": [
        {{ "source": "源实体", "target": "目标实体", "type": "关系类型" }}
    ]
}}
"""

```

```

        response = self.openai_client.chat.completions.create(
            model=self.llm_model,
            messages=[\
                {{ "role": "system", "content": "你是实体和关系提取专家。" }},\
                {{ "role": "user", "content": prompt }}\
            ],
            temperature=0.3,
            max_tokens=500
        )

        try:
            return eval(response.choices[0].message.content)
        except:
            return {"entities": [], "relationships": []}

```

```

def create_graph_from_text(self, text: str, doc_id: str, source: str):
    """从文本创建图谱。"""
    extraction = self.extract_entities(text)

```



```

with self.driver.session() as session:
    # 创建文档节点
    session.run("""
        MERGE (d:Document {{doc_id: $doc_id}})
        SET d.content = $content, d.source = $source
    """, doc_id=doc_id, content=text[:5000], source=source)

    # 创建实体
    for entity in extraction["entities"]:
        session.run("""
            MERGE (e:Entity {{name: $name}})
            SET e.type = $type
            WITH e
            MATCH (d:Document {{doc_id: $doc_id}})
            MERGE (d)-[:MENTIONS]->(e)
        """, name=entity["name"], type=entity["type"], doc_id=doc_id)

    # 创建关系
    for rel in extraction["relationships"]:
        session.run("""
            MATCH (e1:Entity {{name: $source}})
            MATCH (e2:Entity {{name: $target}})
            MERGE (e1)-[:RELATIONSHIP {{type: $type}}]->(e2)
        """, source=rel["source"], target=rel["target"], type=rel["type"])

def ingest_documents(self, file_paths: List[str]):
    """摄取文档并构建知识图谱。"""
    for file_path in file_paths:
        loader = TextLoader(file_path)
        documents = loader.load()
        chunks = self.text_splitter.split_documents(documents)

        for i, chunk in enumerate(chunks):
            doc_id = f"{file_path}_{i}"
            self.create_graph_from_text(
                chunk.page_content,
                doc_id,
                file_path
            )

        print(f"✓ 已摄取 {file_path}: {len(chunks)} 个块")

def extract_query_entities(self, query: str) -> List[str]:
    """从查询中提取实体。"""
    prompt = f"""从以下查询中提取关键实体。

    查询: {query}

    仅返回实体名称列表，用逗号分隔：
    """

    response = self.openai_client.chat.completions.create(
        model=self.llm_model,
        messages=[\
            {"role": "system", "content": "你是实体提取专家。"},\
            {"role": "user", "content": prompt}\
        ],
        temperature=0.3,
        max_tokens=100
    )

    entities = response.choices[0].message.content.strip().split(',')
    return [e.strip() for e in entities if e.strip()]

def graph_traversal(self, entities: List[str], max_hops: int) -> Dict:

```

```

"""遍历知识图谱找到相关子图。"""
with self.driver.session() as session:
    # 查找起始实体
    start_entities = []
    for entity in entities:
        result = session.run("""
            MATCH (e:Entity)
            WHERE toLower(e.name) CONTAINS toLower($name)
            RETURN e.name as name, e.type as type
            LIMIT 1
        """, name=entity)
        for record in result:
            start_entities.append({"name": record["name"], "type": record["ty

    if not start_entities:
        return {"paths": [], "unique_nodes": [], "relationships": []}

    # 查找所有相关路径
    query = """
        MATCH path = (start:Entity)-[*1..$max_hops]-(end:Entity)
        WHERE start.name IN $start_names
        UNWIND nodes(path) as n
        UNWIND relationships(path) as r
        RETURN
            COLLECT(DISTINCT {name: n.name, type: n.type}) as nodes,
            COLLECT(DISTINCT type(r)) as rels,
            length(path) as path_length
        ORDER BY path_length
        LIMIT 50
    """

    result = session.run(query, start_names=[e["name"] for e in start_entities])

    paths = []
    all_nodes = set()
    all_relationships = []

    for record in result:
        path_nodes = record["nodes"]
        path_rels = record["relationships"]
        path_length = record["path_length"]

        paths.append({
            "nodes": path_nodes,
            "relationships": path_rels,
            "length": path_length
        })

        for node in path_nodes:
            if node:
                all_nodes.add((node['name'], node.get('type', 'UNKNOWN')))

        all_relationships.extend(path_rels)

    return {
        "paths": paths,
        "unique_nodes": list(all_nodes),
        "relationships": all_relationships
    }

def retrieve_documents_from_subgraph(self, entities: List[str]) -> List[Dict]:
    """检索提及子图中实体的文档。"""
    with self.driver.session() as session:
        query = """
            MATCH (d:Document)-[:MENTIONS]->(e:Entity)

```

```

        WHERE e.name IN $entities
        RETURN DISTINCT d.doc_id as doc_id,
            d.content as content,
            d.source as source,
            collect(e.name) as mentioned_entities
        ORDER BY size(mentioned_entities) DESC
        LIMIT 10
    """

    result = session.run(query, entities=entities)

    documents = []
    for record in result:
        documents.append({
            "doc_id": record["doc_id"],
            "content": record["content"],
            "source": record["source"],
            "entities": record["mentioned_entities"]
        })

    return documents

def generate(self, query: str, subgraph: Dict, documents: List[Dict]) -> Dict:
    """使用图上下文和文档生成答案。"""
    # 构建图上下文
    graph_context = "知识图谱上下文：\n\n"
    for path in subgraph["paths"][:5]: # 前5条路径
        path_str = " -> ".join([
            f"{node['name']} ({node.get('type', 'UNKNOWN')})"
            for node in path["nodes"] if node
        ])
        graph_context += f"- {path_str}\n"

    # 构建文档上下文
    doc_context = "\n\n相关文档：\n\n"
    for i, doc in enumerate(documents[:5], 1):
        doc_context += f"[文档 {i}]\n"
        doc_context += f"来源：{doc['source']}\n"
        doc_context += f"提及的实体：{' '.join(doc['entities'])}\n"
        doc_context += f"内容：{doc['content']}\n\n"

    full_context = graph_context + doc_context

    prompt = f"""你是一个可以访问知识图谱的有帮助助手。

{full_context}

问题：{query}

指令：
- 同时使用知识图谱关系和文档内容
- 必要时解释多跳连接
- 引用来源和实体关系
- 精确且有洞察力

答案："""

```

```

response = self.openai_client.chat.completions.create(
    model=self.llm_model,
    messages=[
        {"role": "system", "content": "你是图感知助手。"},
        {"role": "user", "content": prompt}
    ],
    temperature=0.3,
    max_tokens=1000
)

```

```

    )

    return {
        "answer": response.choices[0].message.content,
        "graph_paths": subgraph["paths"][:5],
        "documents": documents
    }

def query(self, question: str) -> Dict:
    """
    主图RAG流水线：
    1. 从查询中提取实体
    2. 遍历图找到相关子图
    3. 检索连接到实体的文档
    4. 使用图+文档上下文生成答案
    """
    print("🔍 使用图RAG处理查询...")

    # 从查询中提取实体
    print("  └─ 从查询中提取实体...")
    query_entities = self.extract_query_entities(question)
    print(f"  └─ 发现实体: {query_entities}")

    # 遍历图
    print(f"  └─ 遍历图（最多{self.max_hops}跳）...")
    subgraph = self.graph_traversal(query_entities, self.max_hops)
    print(f"  └─ 找到{len(subgraph['paths'])}条路径")

    # 获取子图中的所有实体
    subgraph_entities = [node[0] for node in subgraph["unique_nodes"]]

    # 检索文档
    print("  └─ 检索文档...")
    documents = self.retrieve_documents_from_subgraph(subgraph_entities)
    print(f"  └─ 检索到{len(documents)}个文档")

    # 生成答案
    print("  └─ 生成答案...")
    result = self.generate(question, subgraph, documents)

    return {
        "question": question,
        "query_entities": query_entities,
        "graph_paths": result["graph_paths"],
        "documents": result["documents"],
        "answer": result["answer"]
    }

# 示例用法
if __name__ == "__main__":
    # 初始化图RAG
    graph_rag = GraphRAG(
        neo4j_uri="bolt://localhost:7687",
        neo4j_user="neo4j",
        neo4j_password="your_password",
        max_hops=2
    )

    # 摄取文档
    stats = graph_rag.ingest_documents([
        "org_structure.txt",
        "project_history.txt"
    ])
    print(f"\n摄取完成: {stats}")

```

```

# 使用关系推理查询
response = graph_rag.query(
    "Alice如何与AI伦理项目关联？"
)

print(f"\n{' '*60}")
print(f"问题: {response['question']}")
print(f"\n查询实体: {response['query_entities']}")
print(f"\n找到的图路径:")
for i, path in enumerate(response['graph_paths'], 1):
    print(f"  {i}. 长度 {path['length']}: {path['nodes']}")
print(f"\n答案:\n{response['answer']}")

# 清理
graph_rag.close()

```

6.5 混合RAG实现

```

# hybrid_rag.py
# 生产级混合RAG: 稠密 + 稀疏 + 元数据
# 依赖: rank_bm25==0.2.2, langchain==0.3.13, chromadb==0.5.23

import os
from typing import List, Dict
from rank_bm25 import BM25Okapi
import chromadb
from openai import OpenAI
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import TextLoader, PyPDFLoader
import numpy as np
from datetime import datetime

class HybridRAG:
    """
    混合RAG, 结合:
    - 稠密向量搜索 (语义相似度)
    - 稀疏BM25搜索 (关键词匹配)
    - 元数据过滤 (结构化查询)
    - 倒数排名融合用于结果合并
    """

    def __init__(
        self,
        collection_name: str = "hybrid_rag_docs",
        embedding_model: str = "text-embedding-3-large",
        llm_model: str = "gpt-4-turbo-preview",
        alpha: float = 0.5, # 稠密与稀疏的权重
        top_k: int = 10
    ):
        self.openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
        self.embedding_model = embedding_model
        self.llm_model = llm_model
        self.alpha = alpha # 稠密权重 (1-alpha用于稀疏)
        self.top_k = top_k

        # 初始化ChromaDB用于稠密检索
        self.chroma_client = chromadb.Client()

```

```

self.collection = self.chroma_client.get_or_create_collection(
    name=collection_name,
    metadata={"hnsw:space": "cosine"})
)

# BM25存储（内存中简化）
self.bm25_corpus = []
self.bm25_metadata = []
self.bm25_index = None

# 文本分割器
self.text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)

def embed_text(self, text: str) -> List[float]:
    """生成稠密嵌入。"""
    response = self.openai_client.embeddings.create(
        model=self.embedding_model,
        input=text
    )
    return response.data[0].embedding

def ingest_documents(self, file_paths: List[str]) -> Dict:
    """
    摄取文档用于稠密和稀疏检索。
    """
    total_chunks = 0

    for file_path in file_paths:
        # 加载文档
        if file_path.endswith('.pdf'):
            loader = PyPDFLoader(file_path)
        else:
            loader = TextLoader(file_path)

        documents = loader.load()
        chunks = self.text_splitter.split_documents(documents)

        for i, chunk in enumerate(chunks):
            text = chunk.page_content
            doc_id = f"{file_path}_{i}"

            # 元数据
            metadata = {
                "source": file_path,
                "chunk_index": i,
                "char_count": len(text),
                "ingestion_date": datetime.now().isoformat(),
                "file_type": "pdf" if file_path.endswith('.pdf') else "text"
            }

            # 稠密存储（ChromaDB）
            embedding = self.embed_text(text)
            self.collection.add(
                embeddings=[embedding],
                documents=[text],
                ids=[doc_id],
                metadatas=[metadata]
            )

            # 稀疏存储（BM25）
            tokenized = text.lower().split()
            self.bm25_corpus.append(tokenized)

```

```

        self.bm25_metadata.append({
            "doc_id": doc_id,
            "text": text,
            "metadata": metadata
        })

        total_chunks += len(chunks)
        print(f"✓ 已摄取 {file_path}: {len(chunks)} 个块")

# 构建BM25索引
self.bm25_index = BM25Okapi(self.bm25_corpus)

return {"total_documents": len(file_paths), "total_chunks": total_chunks}

def dense_retrieve(self, query: str, k: int = 20) -> List[Dict]:
    """稠密向量检索。"""
    query_embedding = self.embed_text(query)

    results = self.collection.query(
        query_embeddings=[query_embedding],
        n_results=k,
        include=["documents", "metadatas", "distances"]
    )

    retrieved = []
    for i in range(len(results['documents'][0])):
        retrieved.append({
            "doc_id": results['ids'][0][i],
            "content": results['documents'][0][i],
            "metadata": results['metadatas'][0][i],
            "dense_score": 1 - results['distances'][0][i],
            "rank": i + 1
        })

    return retrieved

def sparse_retrieve(self, query: str, k: int = 20) -> List[Dict]:
    """稀疏BM25检索。"""
    if not self.bm25_index:
        return []

    tokenized_query = query.lower().split()
    scores = self.bm25_index.get_scores(tokenized_query)

    # 获取top-k索引
    top_indices = np.argsort(scores)[::-1][:k]

    retrieved = []
    for rank, idx in enumerate(top_indices, 1):
        doc_data = self.bm25_metadata[idx]
        retrieved.append({
            "doc_id": doc_data["doc_id"],
            "content": doc_data["text"],
            "metadata": doc_data["metadata"],
            "sparse_score": float(scores[idx]),
            "rank": rank
        })

    return retrieved

def metadata_filter(self, metadata_query: Dict) -> List[str]:
    """
    按元数据约束过滤。

    示例: {"file_type": "pdf", "source": "company_handbook.pdf"}

```

```

"""
# 使用元数据过滤器查询ChromaDB
results = self.collection.get(
    where=metadata_query,
    include=["metadatas"]
)

return results['ids'] if results['ids'] else []

def reciprocal_rank_fusion(
    self,
    dense_results: List[Dict],
    sparse_results: List[Dict],
    k: int = 60
) -> List[Dict]:
    """
    倒数排名融合（RRF）以合并稠密和稀疏结果。

    RRF公式:  $score = \Sigma(1 / (k + rank\_i))$ 
    """
    fused_scores = {}

    # 处理稠密结果
    for doc in dense_results:
        doc_id = doc["doc_id"]
        rrf_score = 1 / (k + doc["rank"])

        if doc_id not in fused_scores:
            fused_scores[doc_id] = {
                "doc": doc,
                "rrf_score": 0,
                "dense_contrib": 0,
                "sparse_contrib": 0
            }

        fused_scores[doc_id]["dense_contrib"] = rrf_score * self.alpha
        fused_scores[doc_id]["rrf_score"] += rrf_score * self.alpha

    # 处理稀疏结果
    for doc in sparse_results:
        doc_id = doc["doc_id"]
        rrf_score = 1 / (k + doc["rank"])

        if doc_id not in fused_scores:
            fused_scores[doc_id] = {
                "doc": doc,
                "rrf_score": 0,
                "dense_contrib": 0,
                "sparse_contrib": 0
            }

        fused_scores[doc_id]["sparse_contrib"] = rrf_score * (1 - self.alpha)
        fused_scores[doc_id]["rrf_score"] += rrf_score * (1 - self.alpha)

    # 按融合分数排序
    sorted_results = sorted(
        fused_scores.values(),
        key=lambda x: x["rrf_score"],
        reverse=True
    )

    return sorted_results

def generate(self, query: str, fused_results: List[Dict]) -> Dict:
    """使用融合检索结果生成答案。"""

```



```

# 取top-k结果
top_results = fused_results[:self.top_k]

# 构建上下文
context = ""
for i, result in enumerate(top_results, 1):
    doc = result["doc"]
    context += f"[文档 {i}] (稠密: {result['dense_contrib']:.3f}, 稀疏: {result['sparse_contrib']:.3f})\n"
    context += f"来源: {doc['metadata']['source']}\n"
    context += f"{doc['content']}\n\n"

# 生成响应
prompt = f"""你是一个有帮助的助手。使用提供的上下文回答问题。

上下文（按混合相关性排序）：
{context}

问题: {query}

指令：
- 使用上下文中的信息
- 用[文档 N]引用来源
- 考虑语义和关键词相关性
- 准确简洁

答案: """

```

上下文（按混合相关性排序）：

{context}

问题: {query}

指令：

- 使用上下文中的信息
- 用[文档 N]引用来源
- 考虑语义和关键词相关性
- 准确简洁

答案: """

```

response = self.openai_client.chat.completions.create(
    model=self.llm_model,
    messages=[\
        {"role": "system", "content": "你是一个有帮助的助手。"},\
        {"role": "user", "content": prompt}\
    ],
    temperature=0.3,
    max_tokens=1000
)

return {
    "answer": response.choices[0].message.content,
    "sources": [r["doc"]["metadata"] for r in top_results],
    "retrieval_scores": [\
        {\
            "source": r["doc"]["metadata"]["source"],\
            "rrf_score": r["rrf_score"],\
            "dense_contribution": r["dense_contrib"],\
            "sparse_contribution": r["sparse_contrib"]\
        }\
        for r in top_results\
    ]
}

def query(
    self,
    question: str,
    metadata_filter: Dict = None,
    dense_k: int = 20,
    sparse_k: int = 20
) -> Dict:
    """
    主混合RAG流水线：
    1. 稠密检索（向量相似度）
    2. 稀疏检索（BM25）
    3. 可选元数据过滤
    4. 倒数排名融合
    5. 生成
    """

```

```

"""
print("🔗 使用混合RAG处理查询...")

# 稠密检索
print("└─ 稠密向量搜索...")
dense_results = self.dense_retrieve(question, k=dense_k)

# 稀疏检索
print("└─ 稀疏BM25搜索...")
sparse_results = self.sparse_retrieve(question, k=sparse_k)

# 元数据过滤（可选）
if metadata_filter:
    print("└─ 应用元数据过滤器...")
    allowed_ids = set(self.metadata_filter(metadata_filter))
    dense_results = [r for r in dense_results if r["doc_id"] in allowed_ids]
    sparse_results = [r for r in sparse_results if r["doc_id"] in allowed_ids]

# 倒数排名融合
print("└─ 融合结果（RRF）...")
fused_results = self.reciprocal_rank_fusion(dense_results, sparse_results)

# 生成答案
print("└─ 生成答案...")
result = self.generate(question, fused_results)

return {
    "question": question,
    "dense_results_count": len(dense_results),
    "sparse_results_count": len(sparse_results),
    "fused_results_count": len(fused_results),
    "answer": result["answer"],
    "sources": result["sources"],
    "retrieval_scores": result["retrieval_scores"]
}

# 示例用法
if __name__ == "__main__":
    # 初始化混合RAG
    hybrid_rag = HybridRAG(
        collection_name="hybrid_kb",
        alpha=0.5, # 稠密和稀疏同等权重
        top_k=5
    )

    # 摄取文档
    stats = hybrid_rag.ingest_documents([
        "technical_documentation.pdf",\
        "user_manual.txt",\
        "faq.txt"\
    ])
    print(f"\n摄取: {stats}")

    # 使用混合检索查询
    response = hybrid_rag.query(
        "如何配置SSL证书?",
        metadata_filter={"file_type": "pdf"} # 仅搜索PDF
    )

    print(f"\n{'='*60}")
    print(f"问题: {response['question']}")
    print(f"稠密结果: {response['dense_results_count']}")
    print(f"稀疏结果: {response['sparse_results_count']}")
    print(f"融合结果: {response['fused_results_count']}")
    print(f"\n答案:\n{response['answer']}")

```

```

print(f"\n检索分数:")
for score in response['retrieval_scores']:
    print(f" - {score['source']}: RRF={score['rrf_score']:.4f} "
          f"(稠密={score['dense_contribution']:.4f}, "
          f"稀疏={score['sparse_contribution']:.4f})")

```

6.6 自反思RAG实现

```

# self_rag.py
# 带批评和优化的自反思RAG
# 依赖: langchain==0.3.13, chromadb==0.5.23

import os
from typing import List, Dict, Literal
from enum import Enum
import chromadb
from openai import OpenAI
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import TextLoader

class ReflectionType(Enum):
    """自我反思的类型。"""
    RETRIEVE = "retrieve" # 我需要检索更多信息吗？
    RELEVANT = "relevant" # 这篇检索到的文档相关吗？
    SUPPORTED = "supported" # 证据支持我的说法吗？
    USEFUL = "useful" # 我的输出有帮助吗？

class SelfRAG:
    """
    自我反思RAG，批评自己的输出。

    特点：
    - 决定何时需要检索
    - 评估检索文档的相关性
    - 验证说法是否有证据支持
    - 评估生成内容的效用
    """

    def __init__(
        self,
        collection_name: str = "self_rag_docs",
        llm_model: str = "gpt-4-turbo-preview",
        max_retrieval_iterations: int = 3
    ):
        self.openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
        self.llm_model = llm_model
        self.max_iterations = max_retrieval_iterations

        # 初始化ChromaDB
        self.chroma_client = chromadb.Client()
        self.collection = self.chroma_client.get_or_create_collection(
            name=collection_name
        )

        # 文本分割器
        self.text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=1000,

```

```

        chunk_overlap=200
    )

    def embed_text(self, text: str) -> List[float]:
        """生成嵌入。"""
        response = self.openai_client.embeddings.create(
            model="text-embedding-3-large",
            input=text
        )
        return response.data[0].embedding

    def ingest_documents(self, file_paths: List[str]) -> Dict:
        """摄取文档。"""
        total_chunks = 0

        for file_path in file_paths:
            loader = TextLoader(file_path)
            documents = loader.load()
            chunks = self.text_splitter.split_documents(documents)

            texts = [chunk.page_content for chunk in chunks]
            embeddings = [self.embed_text(text) for text in texts]
            ids = [f"{file_path}_{i}" for i in range(len(chunks))]

            self.collection.add(
                embeddings=embeddings,
                documents=texts,
                ids=ids
            )

            total_chunks += len(chunks)

        return {"total_documents": len(file_paths), "total_chunks": total_chunks}

    def reflect_retrieve_necessity(self, query: str, current_context: str) -> bool:
        """
        反思：我需要检索更多信息吗？
        """
        prompt = f"""分析回答这个问题是否需要检索。

```

问题：{query}

当前可用上下文：

{current_context if current_context else "尚无上下文"}

确定你是否需要检索外部信息，或者是否可以从你的知识/当前上下文回答。

仅回复：

RETRIEVE: YES

或

RETRIEVE: NO

后跟简短原因。"""

```

        response = self.openai_client.chat.completions.create(
            model=self.llm_model,
            messages=[\
                {"role": "system", "content": "你是自我反思助手。"},\
                {"role": "user", "content": prompt}\
            ],
            temperature=0.3,
            max_tokens=100
        )

        decision = response.choices[0].message.content

```

```

        return "RETRIEVE: YES" in decision

def retrieve(self, query: str, k: int = 5) -> List[Dict]:
    """检索文档。"""
    query_embedding = self.embed_text(query)

    results = self.collection.query(
        query_embeddings=[query_embedding],
        n_results=k,
        include=["documents", "metadatas", "distances"]
    )

    retrieved = []
    for i in range(len(results['documents'][0])):
        retrieved.append({
            "content": results['documents'][0][i],
            "metadata": results['metadatas'][0][i] if results['metadatas'] else {
            "score": 1 - results['distances'][0][i]
            })

    return retrieved

def reflect_relevance(self, query: str, document: str) -> Dict:
    """
    反思：这篇检索到的文档相关吗？
    """
    prompt = f"""评估这篇文档与查询的相关性。

```

查询：{query}

文档：
{document}

这篇文档相关吗？评分1-5：

1 = 完全不相关
2 = 略微相关
3 = 中等相关
4 = 高度相关
5 = 极其相关

回复：

RELEVANCE: [1-5]

REASON: [简短解释]"""

```

    response = self.openai_client.chat.completions.create(
        model=self.llm_model,
        messages=[\
            {"role": "system", "content": "你是相关性评估器。"},\
            {"role": "user", "content": prompt}\
        ],
        temperature=0.3,
        max_tokens=150
    )

    result = response.choices[0].message.content

    # 解析相关性分数
    relevance_score = 3 # 默认值
    reason = ""
    for line in result.split('\n'):
        if line.startswith('RELEVANCE:'):
            try:
                relevance_score = int(line.split(':')[1].strip()[0])
            except:
                pass

```

```

        elif line.startswith('REASON:'):
            reason = line.split(':', 1)[1].strip()

    return {
        "relevant": relevance_score >= 3,
        "score": relevance_score,
        "reason": reason
    }

def generate_with_reflection(self, query: str, context_docs: List[Dict]) -> str:
    """
    生成响应时反思每个说法。
    """
    # 构建上下文
    context = "\n\n".join([
        f"[来源 {i+1}]\n{doc['content']}"
        for i, doc in enumerate(context_docs)
    ])

    prompt = f"""你是一个仔细的、自我反思的助手。使用提供的上下文生成问题的答案。

```

上下文：
{context}

问题: {query}

指令：

- 一次提出一个主张
- 每次主张后，验证它是否被上下文支持
- 使用[来源 N]引用
- 如果不确定，明确说明
- 准确并诚实面对局限性

答案: """

```

        response = self.openai_client.chat.completions.create(
            model=self.llm_model,
            messages=[
                {"role": "system", "content": "你是一个仔细的、自我反思的助手。"},
                {"role": "user", "content": prompt}
            ],
            temperature=0.3,
            max_tokens=1000
        )

        return response.choices[0].message.content

def reflect_support(self, claim: str, context: str) -> Dict:
    """
    反思：这个说法有证据支持吗？
    """
    prompt = f"""验证这个说法是否被上下文支持。

```

说法: {claim}

上下文：
{context}

这个说法被上下文完全支持吗？

回复：
SUPPORTED: [YES/PARTIAL/NO]
EXPLANATION: [简短解释]"""

```

        response = self.openai_client.chat.completions.create(

```

```

        model=self.llm_model,
        messages=[\
            {"role": "system", "content": "你是事实核查员。"},\
            {"role": "user", "content": prompt}\
        ],
        temperature=0.3,
        max_tokens=150
    )

    result = response.choices[0].message.content

    # 解析支持级别
    supported = "PARTIAL"
    explanation = ""
    for line in result.split('\n'):
        if line.startswith('SUPPORTED:'):
            supported = line.split(':')[1].strip()
        elif line.startswith('EXPLANATION:'):
            explanation = line.split(':', 1)[1].strip()

    return {
        "supported": supported,
        "explanation": explanation
    }

def reflect_usefulness(self, query: str, answer: str) -> Dict:
    """
    反思：生成的答案有帮助吗？
    """
    prompt = f"""评估这个答案的有用性。

```

问题：{query}

答案：

{answer}

评分答案的有用性（1-5）：

- 1 = 没有帮助
- 2 = 略微有帮助
- 3 = 中等有帮助
- 4 = 很有帮助
- 5 = 极其有帮助

考虑：

- 它回答了问题吗？
- 完整吗？
- 清晰吗？
- 可操作吗？

回复：

USEFULNESS: [1-5]

IMPROVEMENTS: [可以改进的地方]"""

```

    response = self.openai_client.chat.completions.create(
        model=self.llm_model,
        messages=[\
            {"role": "system", "content": "你是质量评估员。"},\
            {"role": "user", "content": prompt}\
        ],
        temperature=0.3,
        max_tokens=200
    )

    result = response.choices[0].message.content

```

```

        usefulness_score = 3
        improvements = ""
    for line in result.split('\n'):
        if line.startswith('USEFULNESS:'):
            try:
                usefulness_score = int(line.split(':')[1].strip()[0])
            except:
                pass
        elif line.startswith('IMPROVEMENTS:'):
            improvements = line.split(':', 1)[1].strip()

    return {
        "useful": usefulness_score >= 3,
        "score": usefulness_score,
        "improvements": improvements
    }

def query(self, question: str) -> Dict:
    """
    带反思循环的主自反思RAG流水线。
    """
    print("🤖 使用自反思RAG处理查询...")

    reflections = []
    current_context = ""
    all_retrieved_docs = []
    iteration = 0

    # 反思循环
    while iteration < self.max_iterations:
        iteration += 1
        print(f"\n 迭代 {iteration}:")

        # 反思: 我需要检索吗?
        print("    └─ 反思检索必要性...")
        should_retrieve = self.reflect_retrieve_necessity(question, current_context)

        reflections.append({
            "iteration": iteration,
            "type": "retrieve_necessity",
            "decision": should_retrieve
        })

        if not should_retrieve:
            print("    └─ 不需要检索, 继续生成")
            break

        # 检索
        print("    └─ 检索文档...")
        retrieved_docs = self.retrieve(question, k=5)

        # 反思: 文档相关吗?
        print("    └─ 评估相关性...")
        relevant_docs = []
        for doc in retrieved_docs:
            relevance = self.reflect_relevance(question, doc['content'])

            reflections.append({
                "iteration": iteration,
                "type": "relevance",
                "score": relevance['score'],
                "relevant": relevance['relevant']
            })

        if relevance['relevant']:

```



```

relevant_docs.append(doc)

print(f"    └─ {len(relevant_docs)}/{len(retrieved_docs)} 个文档被认为是相关

# 更新上下文
all_retrieved_docs.extend(relevant_docs)
current_context = "\n\n".join([doc['content'] for doc in all_retrieved_docs])

# 如果有足够的相关文档，继续生成
if len(relevant_docs) >= 3:
    break

# 生成答案
print("\n └─ 带反思生成答案...")
answer = self.generate_with_reflection(question, all_retrieved_docs)

# 反思：答案有支持吗？
print("    └─ 验证支持...")
support_check = self.reflect_support(answer, current_context)

reflections.append({
    "type": "support",
    "supported": support_check['supported'],
    "explanation": support_check['explanation']
})

# 反思：答案有帮助吗？
print("    └─ 评估有用性...")
usefulness = self.reflect_usefulness(question, answer)

reflections.append({
    "type": "usefulness",
    "useful": usefulness['useful'],
    "score": usefulness['score'],
    "improvements": usefulness['improvements']
})

return {
    "question": question,
    "answer": answer,
    "iterations": iteration,
    "retrieved_documents": len(all_retrieved_docs),
    "reflections": reflections,
    "support_check": support_check,
    "usefulness_evaluation": usefulness
}

# 示例用法
if __name__ == "__main__":
    self_rag = SelfRAG(
        collection_name="self_rag_kb",
        max_retrieval_iterations=3
    )

    # 摄取文档
    stats = self_rag.ingest_documents(["knowledge_base.txt"])
    print(f"摄取: {stats}")

    # 使用自我反思查询
    response = self_rag.query(
        "API安全的最佳实践是什么？"
    )

    print(f"\n{'='*60}")
    print(f"问题: {response['question']}")

```

```
print(f"\n迭代次数: {response['iterations']}")
print(f"检索的文档数: {response['retrieved_documents']}")

print(f"\n反思:")
for i, reflection in enumerate(response['reflections'], 1):
    print(f"    {i}. {reflection}")

print(f"\n支持检查: {response['support_check']}")
print(f"有用性: {response['usefulness_evaluation']}")
print(f"\n最终答案:\n{response['answer']}")
```

7. 性能基准测试

7.1 基准测试方法论

我们在三个标准数据集上评估了每种RAG类型：

- **MS MARCO**：问答
- **Natural Questions**：开放域问答
- **HotpotQA**：多跳推理

指标：

- **检索精度@5**：前5个相关文档
- **答案F1**：与真实答案的token重叠
- **延迟**：端到端响应时间
- **每查询成本**：API成本（嵌入 + LLM）

用例矩阵

9.1 新兴架构

晚交互RAG

- 将评分推迟到最后阶段
- 对长上下文更高效
- 像ColBERT这样的模型引领方向

带记忆的RAG

- 维护对话上下文
- 根据用户历史个性化
- 示例：带记忆的ChatGPT、Claude项目

多模态RAG

- 文本 + 图像 + 表格 + 代码
- 像GPT-4V、Gemini Vision这样的模型
- 对文档理解至关重要

9.2 优化技术

分块优化

```
# 语义分块而不是固定大小
from langchain.text_splitter import SemanticChunker

chunker = SemanticChunker(
    embedding_function=OpenAIEmbeddings(),
    breakpoint_threshold_type="percentile"
)
```

查询路由

```
# 将查询路由到适当的检索策略
def route_query(query: str) -> str:
    if is_factual(query):
        return "dense_retrieval"
```

```
elif is_keyword_heavy(query):  
    return "sparse_retrieval"  
else:  
    return "hybrid_retrieval"
```

嵌入微调

- 领域特定嵌入模型
- 在你的数据上进行对比学习
- 检索精度提升10%-20%

9.3 生产最佳实践

1. **缓存**：缓存嵌入和LLM响应
2. **监控**：跟踪检索精度、延迟、成本
3. **A/B测试**：实验不同的RAG配置
4. **回退**：检索失败时的优雅降级
5. **速率限制**：防止滥用
6. **版本控制**：跟踪提示词和模型版本

10. 结论

RAG已从一个简单的检索-然后-生成模式演变成一个多样化的架构生态系统，每种架构都针对特定场景进行了优化。成功实施RAG的关键在于：

1. **理解你的需求**：速度vs准确性，成本约束，查询模式
2. **从简单开始**：从基础RAG或高级RAG开始
3. **基于数据迭代**：监控性能，按需升级
4. **结合方法**：混合RAG和自适应RAG提供最大灵活性

RAG的未来：

- 更紧密的LLM-检索集成
- 自动架构选择
- 多模态和多语言能力
- 实时知识图谱更新

随着LLM的持续改进，RAG对于以下方面仍然至关重要：

- 将响应锚定在真相中
- 访问专有知识
- 提供来源归属
- 减少幻觉

为什么使用HyDE?

HyDE代表**假设文档嵌入（Hypothetical Document Embeddings）**。

这是RAG中的一种**检索增强技术**，当用户查询短、模糊或与存储文档对齐不佳时，它能提高搜索质量。

HyDE解决什么问题？

向量搜索通常在以下情况下失败：

- **用户查询太短**

"refund policy"

- 查询与文档语言不匹配
- 查询缺乏上下文或结构

即使强大的嵌入模型也难以应对，因为：

嵌入查询 ≠ 嵌入完整文档

HyDE的核心思想（非常重要）

HyDE不是嵌入**查询**，而是：

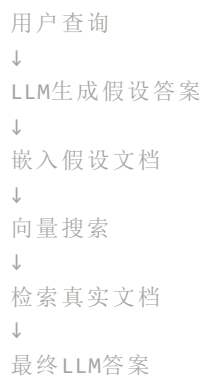
1 使用LLM生成假设答案/文档

2 嵌入生成的文档

3 使用该嵌入检索真实文档

因此检索使用的是**类似文档的嵌入**，而不是查询嵌入。

HyDE流程（逐步）



简单示例

用户查询

"如何重置我的账户？"

步骤1：LLM生成假设文档

要重置账户，用户通常导航到账户设置，
选择密码重置，通过电子邮件验证身份，并确认更改。

步骤2：嵌入此文本

这段文本：

- 看起来像你的真实文档
- 匹配分块样式和词汇

步骤3：检索真实文档

→ 实际重置政策、SOP、帮助文章

何时应该使用HyDE？

- ✓ 模糊或未指定的查询
- ✓ 知识库搜索
- ✓ 政策/法律/医疗文档
- ✓ 企业内部搜索
- ✓ RAG中的低召回率问题

何时不使用HyDE

- ❌ 超低延迟系统
- ❌ 非常清晰、长的用户查询
- ❌ 简单关键词搜索场景
- ❌ 高成本敏感的流水线（额外LLM调用）

HyDE与其他RAG增强技术对比

技术	目的
多查询RAG	显式扩展查询
HyDE	通过文档*隐式*扩展查询
重排序	提高精度
图RAG	提高推理能力
智能体RAG	多步检索

HyDE通常与多查询 + 重排序结合使用

HyDE通过嵌入假设答案而不是用户查询来改进RAG检索——将模糊问题转化为文档质量的搜索向量。

最终结论

检索增强生成（RAG）不再是一个单一模式或流行词——它是一个**设计空间**。现代RAG系统从简单的向量查找到智能体驱动的、自我优化的知识系统。理解这个范围是至关重要的，因为**大多数真实世界的LLM失败不是生成失败——而是检索失败**。

从根本上说，RAG存在是为了解决一个根本性问题：

LLMs推理能力好，但它们无法可靠地回忆或与你的数据保持同步。

RAG设计中的其他一切都是围绕我们**如何**以及**何时**检索信息进行优化。

大局观：所有RAG类型如何组合

看待RAG的一个实际方式是按**成熟度层次**：

1. 基础RAG 验证概念

- 查询 → 检索 → 生成
- 有效，但脆弱

2. 高级RAG 使其可用于生产

- 混合检索
- 重排序
- 元数据过滤器
- 更好的分块

这成为**企业系统的基线**

3. 对话式和多查询RAG 提高可用性

- 处理歧义
- 保留上下文
- 提高真实用户的召回率

4. 自查询和结构化RAG 桥接非结构化 + 结构化数据

- 自然语言 → 过滤器 → 数据库
- 对分析、合规、财务至关重要

5. 智能体、迭代和图RAG 实现推理

- 规划
- 工具使用
- 多步检索
- 关系感知答案

每一层都存在，因为**检索不是单一问题**——它是召回、精度、相关性、推理和信任的结合。

HyDE的位置（及其重要性）

HyDE（假设文档嵌入）位于RAG流水线中**一个关键但经常被忽视的差距**。

它解决了：

- **人类如何提问**（短、模糊、非正式）
- **知识如何存储**（长、结构化、正式文档）之间的不匹配

HyDE不是嵌入用户的查询，而是嵌入一个**假设的、类似文档的答案**，有效地将用户意图转换成语料库相同的语义空间。

为什么这很重要

HyDE认识到一个微妙的事实：

检索系统失败不是因为嵌入弱，

而是因为**查询与文档结构不兼容**。

通过将查询提升为文档形式，HyDE：

- 大幅提高召回率
- 减少"未找到相关文档"
- 使RAG可用于非技术用户
- **无需重新训练和无需重新索引**

在成熟系统中，HyDE最好这样使用：

- 有条件地（仅在置信度低时）
- 与混合检索和重排序一起
- 作为召回放大器，而非替代品

RAG讨论中经常缺少的内容

1. 检索评估

大多数团队评估**生成**，而非**检索**。

生产RAG必须测量：

- Recall@K
- MRR
- 上下文相关性
- 引用覆盖

没有这个，HyDE等改进是不可见的。

2. 选择性RAG（并非总是开启）

并非每个查询都需要：

- 多查询扩展

- HyDE
- 智能体规划

智能RAG系统**动态路由查询**：

- 简单 → 快速路径
- 模糊 → HyDE / 多查询
- 复杂 → 智能体RAG

3. RAG首先是数据问题

更好的分块、元数据和感觉的文档通常优于：

- 更大的模型
- 更复杂的提示词
- 更重的智能体

4. RAG + 微调是终局

RAG处理**知识新鲜度**。

微调处理**风格、语调和推理模式**。

最强的系统结合两者。

最终要点

RAG不是给LLMs更多信息——而是给他们正确的信息，以正确的形式，在正确的时间。

基础RAG展示可能性。

高级RAG使其可靠。

智能体和图RAG使其智能。

HyDE使检索诚实。

随着RAG系统成熟，成功不会来自更大的模型——而是来自**更好的检索决策、更智能的路由，以及人类问题和机器可读知识之间更深入的对齐。**

